

how can i connect this login page to my database and keep certain items "hidden" until logged in?:

```
return (
  <>
    <main>

      <Navbar />

      <Background />

      <section className={styles.loginShell}>
        <div className={styles.loginCard}>
          <div className={styles.loginHeader}>
            <h1>Welcome Back</h1>

            <p>Sign in to access your Campus Companion
account.</p>
          </div>

          <form className={styles.loginForm} onSubmit=
{handleLogin}>
            <div className={styles.formGroup}>
              <label htmlFor="studentId">Student
ID</label>

              <input id="studentId" type="text"
placeholder="Enter your student ID" value={studentId}
onChange={(e) => setStudentId(e.target.value)}>
            </div>

            <div className={styles.formGroup}>
              <label htmlFor="password">Password</label>

              <input id="password" type="password"
placeholder="Enter your password" value={password}
onChange={(e) => setPassword(e.target.value)}>
            </div>

            {error && <p className={styles.errorMessage}>
{error}</p>}
            {success && <p className=
{styles.successMessage}>{success}</p>}

            <button type="submit" className=
```

```

{styles.loginButton} disabled={loading}>{loading ?
"Signing In..." : "Sign In"}</button>

</form>

<div className={styles.loginFooter}>

  <p>Demo login: <strong>ABC12345</strong>

</p>

  <p>Password: <strong>campus123</strong>

</p>

  <Link href="/" className=
{styles.backLink}>Back to Home</Link>

</div>
</div>
</section>
</main>
</>
)

```

Thought for a couple of seconds

You need 3 parts:

1. A database table for users
2. A server-side login API that checks the database
3. Conditional rendering / protected routes so hidden items only show after login

Assuming this is Next.js + React, here's the usual pattern.

1. Do not connect directly from the login page to the database

Your React page should **not** query the database directly because that would expose credentials. Instead, submit the login form to an API route.

Example `handleLogin` :

```

async function handleLogin(e) {
  e.preventDefault();
  setLoading(true);
  setError("");
  setSuccess("");

  const res = await fetch("/api/login", {

```

```

    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ studentId, password }),
  });

  const data = await res.json();

  if (!res.ok) {
    setError(data.message || "Login failed");
    setLoading(false);
    return;
  }

  setSuccess("Logged in successfully!");
  window.location.href = "/dashboard";
}

```

2. Create a login API route

Example using Next.js App Router:

app/api/login/route.js

```

import { NextResponse } from "next/server";
import bcrypt from "bcryptjs";
import { cookies } from "next/headers";
import { SignJWT } from "jose";
import { db } from "@/lib/db";

const secret = new TextEncoder().encode(process.env.JWT_SECRET);

export async function POST(req) {
  const { studentId, password } = await req.json();

  const user = await db.user.findUnique({
    where: { studentId },
  });

  if (!user) {
    return NextResponse.json(
      { message: "Invalid student ID or password" },
      { status: 401 }
    );
  }

  const passwordMatches = await bcrypt.compare(password, user.passwordHash);

  if (!passwordMatches) {
    return NextResponse.json(
      { message: "Invalid student ID or password" },
      { status: 401 }
    );
  }
}

```

```

    });
  }

  const token = await new SignJWT({
    userId: user.id,
    studentId: user.studentId,
  })
    .setProtectedHeader({ alg: "HS256" })
    .setExpirationTime("1d")
    .sign(secret);

  cookies().set("authToken", token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === "production",
    sameSite: "strict",
    path: "/",
    maxAge: 60 * 60 * 24,
  });

  return NextResponse.json({ message: "Login successful" });
}

```

You would need these packages:

Bash

And your `.env` :

```

JWT_SECRET="some-long-random-secret"
DATABASE_URL="your database url"

```

3. Hide items until logged in

For simple UI hiding, use state:

```

{isLoggedIn && (
  <Link href="/dashboard">Dashboard</Link>
)}

```

But the safer way is to check the login cookie on the server.

Example helper:

```

import { cookies } from "next/headers";
import { jwtVerify } from "jose";

```

```

const secret = new TextEncoder().encode(process.env.JWT_SECRET);

export async function getUserFromSession() {
  const token = cookies().get("authToken")?.value;

  if (!token) return null;

  try {
    const { payload } = await jwtVerify(token, secret);
    return payload;
  } catch {
    return null;
  }
}

```

Then in a server component like your navbar:

```

import Link from "next/link";
import { getUserFromSession } from "@/lib/auth";

export default async function Navbar() {
  const user = await getUserFromSession();

  return (
    <nav>
      <Link href="/">Home</Link>

      {user && <Link href="/dashboard">Dashboard</Link>}
      {user && <Link href="/profile">Profile</Link>}

      {!user && <Link href="/login">Login</Link>}
    </nav>
  );
}

```

4. Protect a page completely

For example, `app/dashboard/page.jsx` :

```

import { redirect } from "next/navigation";
import { getUserFromSession } from "@/lib/auth";

export default async function DashboardPage() {
  const user = await getUserFromSession();

  if (!user) {
    redirect("/login");
  }
}

```

```

    return (
      <main>
        <h1>Dashboard</h1>
        <p>Only logged-in users can see this.</p>
      </main>
    );
  }
}

```

Important

Do **not** keep this in production:

```

<p>Demo login: <strong>ABC12345</strong></p>
<p>Demo password: <strong>campus123</strong></p>

```

Also do not store plain passwords in your database. Store only a hashed password like:

JavaScript

The basic flow is:

Login form → /api/login → check database → set secure cookie → show protected items



```

i want to keep this timetable page hidden: <div
className={styles.timetableContentColumn}>
  <section className={ ${styles.section}
${styles.timetablePanel} }>
    <h2 className={styles.timetableHeading}>Weekly
Schedule</h2>

```

```

    <p className=
{styles.timetableSubheading}>Select a day to view
scheduled classes, locations and delivery details.</p>

```

```

    {authLoading && (
      <div className={styles.messageCard}>Checking
login status...</div>
    )}

```

```

    {!authLoading && !user && (

```

```
        <div className={styles.messageCard}>Please  
login to view timetable information.</div>  
    )}
```

```
    {!authLoading && user && (  
    <>  
        <div className={styles.dayTabs} role="tablist"  
aria-label="Select timetable day">
```

```
            {DAYS.map((day) => (  
                <button key={day} type="button" role="tab"  
aria-selected={selectedDay === day} className=  
{`${styles.dayButton}`  
                ${selectedDay === day ?  
styles.activeDayButton : ""} } onClick={() =>  
setSelectedDay(day)}>{day}</button>  
            )})  
        </div>
```

```
        <h3 className={styles.visuallyHidden}>  
{pageTitle}</h3>
```

```
        {loading && (  
            <div className={styles.messageCard}>Loading  
timetable...</div>  
        )}
```

```
        {error && (  
            <div className={styles.errorCard}  
role="alert">{error}</div>  
        )}
```

```
        {!loading && !error && entries.length === 0  
&& (  
            <div className={styles.messageCard}>No  
Classes Scheduled for {selectedDay}.</div>  
        )}
```

```
        {!loading && !error && entries.length > 0 && (  
            <div className={styles.tableCard}>  
                <div className={styles.tableWrapper}>  
                    <table className={styles.table}>  
                        <caption className=  
{styles.visuallyHidden}>Timetable entries for {selectedDay}  
</caption>
```

```

<thead>
  <tr>

    <th scope="col">Time</th>
    <th scope="col">Module</th>
    <th scope="col">Location</th>
    <th scope="col">Lecturer</th>
    <th scope="col">Mode</th>
    <th scope="col">Week</th>

  </tr>
</thead>

<tbody>
  {entries.map((entry) => (
    <tr key={entry.id}>
      <td className={styles.timeCell}>
{formatTime(entry.start_time)} -{" "}
{formatTime(entry.end_time)}</td>

      <td>
        <div className=
{styles.moduleName}>{entry.module_name}</div>

        <div className=
{styles.moduleCode}>{entry.module_code}</div>
      </td>

      <td>{entry.location}</td>

      <td>{entry.lecturer_name}</td>

      <td><span className=
{styles.modeBadge}>{entry.delivery_mode}</span>
      </td>

      <td>
        <span className=
{styles.weekBadge}>{entry.week_type}</span>
      </td>
    </tr>
  )})
</tbody>
</table>
</div>
</div>

```



```

    })
  </>
})
</section>

<section className={styles.infoStrip}>
  <div className={styles.infoCard}>
    <h3>Fictional data only</h3>

    <p>All timetable entries, locations, lecturer
names and modules are fictional for coursework use.</p>
  </div>

  <div className={styles.infoCard}>
    <h3>Placeholder</h3>

    <p>Placeholder</p>
  </div>

  <div className={styles.infoCard}>
    <h3>Placeholder</h3>

    <p>Placeholder</p>
  </div>

</section>

</div>

</main>

```

Wrap the **whole timetable page content** behind your login check, not just the table.

Right now, only the day tabs/table are hidden. This part still shows:

```

<h2>Weekly Schedule</h2>
<p>Select a day...</p>
<section className={styles.infoStrip}>

```

Use this structure instead:

```

<div className={styles.timetableContentColumn}>
  {authLoading && (

```

```

<section className={` ${styles.section} ${styles.timetablePanel}`}>
  <div className={styles.messageCard}>Checking login status...</div>
</section>
)}

{!authLoading && !user && (
  <section className={` ${styles.section} ${styles.timetablePanel}`}>
    <div className={styles.messageCard}>
      Please login to view timetable information.
    </div>
  </section>
)}

{!authLoading && user && (
  <>
    <section className={` ${styles.section} ${styles.timetablePanel}`}>
      <h2 className={styles.timetableHeading}>Weekly Schedule</h2>

      <p className={styles.timetableSubheading}>
        Select a day to view scheduled classes, locations and delivery details.
      </p>

      <div
        className={styles.dayTabs}
        role="tablist"
        aria-label="Select timetable day"
      >
        {DAYS.map((day) => (
          <button
            key={day}
            type="button"
            role="tab"
            aria-selected={selectedDay === day}
            className={` ${styles.dayButton} ${
              selectedDay === day ? styles.activeDayButton : ""
            }`}
            onClick={() => setSelectedDay(day)}
          >
            {day}
          </button>
        ))}
      </div>

      <h3 className={styles.visuallyHidden}>{pageTitle}</h3>

      {loading && (
        <div className={styles.messageCard}>Loading timetable...</div>
      )}

      {error && (
        <div className={styles.errorCard} role="alert">
          {error}
        </div>
      )}

      {!loading && !error && entries.length === 0 && (

```

```

<div className={styles.messageCard}>
  No Classes Scheduled for {selectedDay}.
</div>
)}

{!loading && !error && entries.length > 0 && (
  <div className={styles.tableCard}>
    <div className={styles.tableWrapper}>
      <table className={styles.table}>
        <caption className={styles.visuallyHidden}>
          Timetable entries for {selectedDay}
        </caption>

        <thead>
          <tr>
            <th scope="col">Time</th>
            <th scope="col">Module</th>
            <th scope="col">Location</th>
            <th scope="col">Lecturer</th>
            <th scope="col">Mode</th>
            <th scope="col">Week</th>
          </tr>
        </thead>

        <tbody>
          {entries.map((entry) => (
            <tr key={entry.id}>
              <td className={styles.timeCell}>
                {formatTime(entry.start_time)} - {formatTime(entry.end_time)}
              </td>

              <td>
                <div className={styles.moduleName}>
                  {entry.module_name}
                </div>
                <div className={styles.moduleCode}>
                  {entry.module_code}
                </div>
              </td>

              <td>{entry.location}</td>
              <td>{entry.lecturer_name}</td>

              <td>
                <span className={styles.modeBadge}>
                  {entry.delivery_mode}
                </span>
              </td>

              <td>
                <span className={styles.weekBadge}>
                  {entry.week_type}
                </span>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </div>
)}

```

```

        </tbody>
      </table>
    </div>
  </div>
)}
</section>

<section className={styles.infoStrip}>
  <div className={styles.infoCard}>
    <h3>Fictional data only</h3>
    <p>
      All timetable entries, locations, lecturer names and modules are
      fictional for coursework use.
    </p>
  </div>

  <div className={styles.infoCard}>
    <h3>Placeholder</h3>
    <p>Placeholder</p>
  </div>

  <div className={styles.infoCard}>
    <h3>Placeholder</h3>
    <p>Placeholder</p>
  </div>
</section>
</>
)}
</div>

```

The key change is this:

```

{!authLoading && user && (
  <>
    { /* full timetable content goes here */ }
  </>
)}

```

So only logged-in users can see the timetable heading, tabs, table, and info cards.



Also, how can i keep buttons in my side menu "hidden" until after login?

Same idea as your timetable — you conditionally render based on whether a `user` exists.

1. Basic approach (client-side sidebar)

Wherever your sidebar/menu lives, wrap the buttons:

```
export default function Sidebar({ user }) {
  return (
    <aside className={styles.sidebar}>
      <button>Home</button>

      {user && <button>Timetable</button>}
      {user && <button>Dashboard</button>}
      {user && <button>Profile</button>}

      {!user && <button>Login</button>}
    </aside>
  );
}
```

So:

- `user === null` → only public buttons show
- `user !== null` → protected buttons appear

2. If your sidebar doesn't have `user` yet

You need to get auth state into it.

Option A (best in Next.js App Router): server component

```
import { getUserFromSession } from "@lib/auth";

export default async function Sidebar() {
  const user = await getUserFromSession();

  return (
    <aside>
      <button>Home</button>

      {user && <button>Timetable</button>}
      {user && <button>Dashboard</button>}

      {!user && <button>Login</button>}
    </aside>
  );
}
```

This is the cleanest because it's secure and runs on the server.

Option B: client-side (using state)

If your sidebar is a client component:

```
"use client";

import { useEffect, useState } from "react";

export default function Sidebar() {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("/api/me")
      .then((res) => res.json())
      .then((data) => {
        setUser(data.user);
        setLoading(false);
      });
  }, []);

  if (loading) return null; // or spinner

  return (
    <aside>
      <button>Home</button>

      {user && <button>Timetable</button>}
      {user && <button>Dashboard</button>}

      {!user && <button>Login</button>}
    </aside>
  );
}
```

3. Important (don't skip this)

Hiding buttons ≠ security.

Even if you hide the button, a user could still go to:

```
/timetable
```

So you must also protect the page itself:

```
if (!user) {
  redirect("/login");
}
```

4. Cleaner pattern (recommended)

Instead of repeating `user` && everywhere:

```
function Protected({ user, children }) {  
  if (!user) return null;  
  return children;  
}
```

Use it like:

```
<Protected user={user}>  
  <button>Timetable</button>  
  <button>Dashboard</button>  
</Protected>
```

Summary

- Sidebar buttons:

JavaScript

- Get user from:
 - `server ( getUserFromSession )`  best
 - or `API ( /api/me )` on client
- Always protect actual pages too (redirect if not logged in)

If you want, I can wire your sidebar + login + timetable into one clean auth flow so everything stays in sync.



can you give me what i need to make this settings page work and explain it to me in detail?: i want everything to be "previewable" after clicking on the option BUT only

save to the database AFTER clicking the save button. It also needs to only save AFTER logging in: return (

```
<main className={styles.settingsPage}>
```

```
<Navbar />
```

```
<Background />
```

```
<div className={styles.AccountInfoCard}>
```

```
<div className={styles.AccountInfoContent}>
```

```
<h2>Account Information:</h2>
```

```
<p><strong>Name:</strong> {user?.name || "N/A"}</p>
```

```
<p><strong>Student ID:</strong> {user?.studentId || "N/A"}</p>
```

```
<p><strong>Course:</strong> {user?.course || "N/A"}</p>
```

```
<button type="button" className={styles.secondaryButton} onClick={handleSignOut} aria-label="Sign Out">Sign Out</button>
```

```
</div>
```

```
</div>
```

```
<div className={styles.summaryCard}>
```

```
<div className={styles.summaryCardContent}>
```

```
<h2 className={styles.summaryTitle}>Current Summary:</h2>
```

```
<p><strong>Large Text:</strong> {largeText ? "On" : "Off"}</p>
```

```
<p><strong>High Contrast:</strong> {highContrast ? "On" : "Off"}</p>
```

```
<p><strong>Reduced Motion:</strong> {reducedMotion ? "On" : "Off"}</p>
```

```
<p><strong>Event Reminders:</strong> {eventReminders ? "On" : "Off"}</p>
```

```
<p><strong>Society Alerts:</strong> {societyAlerts ? "On" : "Off"}</p>
```



```
<p><strong>Helpdesk Updates:</strong>
{helpdeskUpdates ? "On" : "Off"}</p>
```

```
<p><strong>Default Page:</strong> {defaultPage}
</p>
```

```
<div className={styles.quickLinks}>
  <button type="button" className=
{styles.quickLinkButton} onClick={() => setLargeText(true)}
aria-label="Accessibility Help">Accessibility
Help</button>
```

```
  <button type="button" className=
{styles.quickLinkButton} onClick={() =>
setEventReminders(true)} aria-label="Notification
Help">Notification Help</button>
```

```
  <button type="button" className=
{styles.quickLinkButton} onClick={handleOpenDefaultPage}
aria-label="Open Default Page">Open Default
Page</button>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div className={styles.contentLayer}>
  <section className={styles.settingsHero}>
    <div className={styles.settingsHeroContent}>
      <div>
        <h1>Settings</h1>
```

```

        <p>Adjust accessibility, notifications and personal
preferences for your Campus Companion experience.</p>
```

```
      </div>
```

```
    </div>
```

```
  </section>
```

```
  <section className={styles.settingsPanel}>
    <h2 className=
{styles.settingsHeading}>Preferences:</h2>
```

```
    <div className={styles.settingsGrid}>
```

```
      <div className={styles.settingsCard}>
```

```
        <h3>Accessibility</h3>
```

```

        <label className={styles.toggleRow}>
```

Large text

```
<input type="checkbox" checked={largeText}
onChange={() => setLargeText((current) => !current)} aria-
label="Set Large Text"/>
</label>
```

```
<label className={styles.toggleRow}>
  <span>High Contrast Mode</span>
```

```
<input type="checkbox" checked=
{highContrast} onChange={() => setHighContrast((current)
=> !current)} aria-label="Set High Contrast"/>
</label>
```

```
<label className={styles.toggleRow}>
  <span>Reduced Motion</span>
```

```
<input type="checkbox" checked=
{reducedMotion} onChange={() =>
setReducedMotion((current) => !current)} aria-label="Set
Reduced Motion"/>
</label>
```

</div>

```
<div className={styles.settingsCard}>
  <h3>Notifications</h3>
```

```
<label className={styles.toggleRow}>
  <span>Event Reminders</span>
```

```
<input type="checkbox" checked=
{eventReminders} onChange={() =>
setEventReminders((current) => !current)} aria-
label="Enable Event Reminders"/>
</label>
```

```
<label className={styles.toggleRow}>
  <span>Society Alerts</span>
```

```
<input type="checkbox" checked={societyAlerts}
onChange={() => setSocietyAlerts((current) => !current)}
aria-label="Enable Society Alerts"/>
</label>
```

```
<label className={styles.toggleRow}>
```

```
<span>Helpdesk Updates</span>
```

```
    <input type="checkbox" checked=
{helpdeskUpdates} onChange={() =>
setHelpdeskUpdates((current) => !current)} aria-
label="Enable Help Desk Updates"/>
  </label>
</div>
```

```
<div className={styles.settingsCard}>
  <h3>App Preferences</h3>
```

```
  <label className={styles.selectGroup}>
    <span>Default Page</span>
```

```
    <select value={defaultPage} onChange={(e) =>
setDefaultPage(e.target.value as DefaultPage)} className=
{styles.selectInput} aria-label="Change Default Page">
```

```
    <option>Campus Map</option>
```

```
    <option>Timetable</option>
```

```
    <option>Societies</option>
```

```
    <option>Helpdesk</option>
```

```
  </select>
```

```
</label>
```

```
<div className={styles.statusArea}>
  {loadingSettings && (
    <p className={styles.statusMessage}>Loading
saved settings...</p>
  )}
```

```
  {!loadingSettings && saveMessage && (
    <p className={styles.statusMessage2}
role="status">{saveMessage}</p>
  )}
</div>
```

```
<div className={styles.buttonRow}>
  <button type="button" className=
{styles.saveButton} onClick={handleSaveSettings}
disabled={savingSettings}
  aria-label="Save Setting Configuration">
{savingSettings ? "Saving..." : "Save Settings"}</button>
```

```

        <button type="button" className=
{styles.restoreButton} onClick={handleRestoreDefaults}
disabled={savingSettings}
        aria-label="Retore Default Settings">Restore
Defaults</button>
    </div>
</div>
</div>
</section>
</div>
</main>
);

```

I'll show you the full pattern: local "draft" settings for instant preview, saved settings loaded from the database, and a protected API so only logged-in users can save.

Thought for a couple of seconds

Yes. The key idea is:

```

Database settings = savedSettings
Page state = previewSettings
Click option = update previewSettings only
Click Save = send previewSettings to database
Not logged in = cannot save

```

Below is a complete setup you can adapt.

1. Database table

If you are using Prisma, add settings fields to your `User` model.

```

model User {
  id          String @id @default(cuid())
  name        String
  studentId   String @unique
  course      String?
  passwordHash String

  largeText    Boolean @default(false)
  highContrast Boolean @default(false)
  reducedMotion Boolean @default(false)

  eventReminders Boolean @default(true)
  societyAlerts   Boolean @default(true)
  helpdeskUpdates Boolean @default(true)

```

```
    defaultPage      String  @default("Campus Map")
  }
}
```

Then run:

Bash

2. Create a protected settings API

Create:

app/api/settings/route.ts

```
import { NextResponse } from "next/server";
import { db } from "@/lib/db";
import { getUserFromSession } from "@/lib/auth";

const allowedDefaultPages = [
  "Campus Map",
  "Timetable",
  "Societies",
  "Helpdesk",
];

export async function GET() {
  const sessionUser = await getUserFromSession();

  if (!sessionUser) {
    return NextResponse.json(
      { message: "You must be logged in." },
      { status: 401 }
    );
  }

  const user = await db.user.findUnique({
    where: {
      id: sessionUser.userId,
    },
    select: {
      name: true,
      studentId: true,
      course: true,
      largeText: true,
      highContrast: true,
      reducedMotion: true,
      eventReminders: true,
    },
  });
}
```

```

        societyAlerts: true,
        helpdeskUpdates: true,
        defaultPage: true,
    },
});

if (!user) {
    return NextResponse.json(
        { message: "User not found." },
        { status: 404 }
    );
}

return NextResponse.json({ user });
}

export async function PATCH(req: Request) {
    const sessionUser = await getUserFromSession();

    if (!sessionUser) {
        return NextResponse.json(
            { message: "You must be logged in before saving settings." },
            { status: 401 }
        );
    }

    const body = await req.json();

    if (!allowedDefaultPages.includes(body.defaultPage)) {
        return NextResponse.json(
            { message: "Invalid default page." },
            { status: 400 }
        );
    }

    const updatedUser = await db.user.update({
        where: {
            id: sessionUser.userId,
        },
        data: {
            largeText: Boolean(body.largeText),
            highContrast: Boolean(body.highContrast),
            reducedMotion: Boolean(body.reducedMotion),
            eventReminders: Boolean(body.eventReminders),
            societyAlerts: Boolean(body.societyAlerts),
            helpdeskUpdates: Boolean(body.helpdeskUpdates),
            defaultPage: body.defaultPage,
        },
        select: {
            name: true,
            studentId: true,
            course: true,
            largeText: true,
            highContrast: true,
            reducedMotion: true,
            eventReminders: true,

```

```

        societyAlerts: true,
        helpdeskUpdates: true,
        defaultPage: true,
      },
    });

    return NextResponse.json({
      message: "Settings saved.",
      user: updatedUser,
    });
  }
}

```

This does two important things:

TypeScript

loads saved settings from the database.

TypeScript

saves new settings, but only after checking:

TypeScript

So a user cannot save settings unless logged in.

3. Your settings page state

At the top of your settings page, use one object for the preview settings.

```

"use client";

import { useEffect, useState } from "react";
import { useRouter } from "next/navigation";

type DefaultPage = "Campus Map" | "Timetable" | "Societies" | "Helpdesk";

type Settings = {
  largeText: boolean;
  highContrast: boolean;
  reducedMotion: boolean;
  eventReminders: boolean;
  societyAlerts: boolean;
}

```

```

    helpdeskUpdates: boolean;
    defaultPage: DefaultPage;
  };

  type User = {
    name: string;
    studentId: string;
    course: string | null;
  };

  const defaultSettings: Settings = {
    largeText: false,
    highContrast: false,
    reducedMotion: false,
    eventReminders: true,
    societyAlerts: true,
    helpdeskUpdates: true,
    defaultPage: "Campus Map",
  };

```

Then inside your component:

```

const router = useRouter();

const [user, setUser] = useState<User | null>(null);

const [settings, setSettings] = useState<Settings>(defaultSettings);
const [savedSettings, setSavedSettings] = useState<Settings>(defaultSettings);

const [loadingSettings, setLoadingSettings] = useState(true);
const [savingSettings, setSavingSettings] = useState(false);
const [saveMessage, setSaveMessage] = useState("");

```

The important part:

TypeScript

is the preview version.

TypeScript

is what is currently saved in the database.

4. Load saved settings after login


```

useEffect(() => {
  async function loadSettings() {
    try {
      const res = await fetch("/api/settings");

      if (res.status === 401) {
        setUser(null);
        setLoadingSettings(false);
        return;
      }

      const data = await res.json();

      if (!res.ok) {
        throw new Error(data.message || "Failed to load settings.");
      }

      const loadedSettings: Settings = {
        largeText: data.user.largeText,
        highContrast: data.user.highContrast,
        reducedMotion: data.user.reducedMotion,
        eventReminders: data.user.eventReminders,
        societyAlerts: data.user.societyAlerts,
        helpdeskUpdates: data.user.helpdeskUpdates,
        defaultPage: data.user.defaultPage,
      };

      setUser({
        name: data.user.name,
        studentId: data.user.studentId,
        course: data.user.course,
      });

      setSettings(loadedSettings);
      setSavedSettings(loadedSettings);
    } catch (error) {
      setSaveMessage("Could not load settings.");
    } finally {
      setLoadingSettings(false);
    }
  }

  loadSettings();
}, []);

```

5. Make option clicks preview only

Instead of separate states like `largeText` , `highContrast` , etc., update the `settings` object.

Example:

```
<input
  type="checkbox"
  checked={settings.largeText}
  onChange={() =>
    setSettings((current) => ({
      ...current,
      largeText: !current.largeText,
    }))
  }
/>
```

This changes the page immediately, but it does **not** save to the database yet.

6. Preview the visual changes immediately

Add this:

```
useEffect(() => {
  document.body.classList.toggle("largeText", settings.largeText);
  document.body.classList.toggle("highContrast", settings.highContrast);
  document.body.classList.toggle("reducedMotion", settings.reducedMotion);
}, [settings.largeText, settings.highContrast, settings.reducedMotion]);
```

Then in your global CSS:

```
body.largeText {
  font-size: 1.15rem;
}

body.highContrast {
  background: #000;
  color: #fff;
}

body.highContrast button,
body.highContrast input,
body.highContrast select {
  border: 2px solid #fff;
}

body.reducedMotion *,
body.reducedMotion *::before,
body.reducedMotion *::after {
  animation: none !important;
}
```

```
    transition: none !important;
  }
}
```

Now when someone clicks "Large Text" or "High Contrast", they see the change instantly.

But it is still only a preview.

7. Save only when clicking Save

```
async function handleSaveSettings() {
  if (!user) {
    setSaveMessage("Please log in before saving settings.");
    return;
  }

  setSavingSettings(true);
  setSaveMessage("");

  try {
    const res = await fetch("/api/settings", {
      method: "PATCH",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(settings),
    });

    const data = await res.json();

    if (!res.ok) {
      throw new Error(data.message || "Failed to save settings.");
    }

    setSavedSettings(settings);
    setSaveMessage("Settings saved successfully.");
  } catch (error) {
    setSaveMessage("Settings could not be saved.");
  } finally {
    setSavingSettings(false);
  }
}
```

8. Restore defaults without saving immediately

This should preview defaults, but not save them yet:

```
function handleRestoreDefaults() {  
  setSettings(defaultSettings);  
  setSaveMessage("Defaults previewed. Click Save Settings to store them.");  
}
```

So restore works like:

Click Restore Defaults → preview defaults

Click Save Settings → save defaults to database

9. Sign out

```
async function handleSignOut() {  
  await fetch("/api/logout", {  
    method: "POST",  
  });  
  
  router.push("/login");  
}
```

10. Open default page

```
function handleOpenDefaultPage() {  
  const routes: Record<DefaultPage, string> = {  
    "Campus Map": "/campus-map",  
    Timetable: "/timetable",  
    Societies: "/societies",  
    Helpdesk: "/helpdesk",  
  };  
  
  router.push(routes[settings.defaultPage]);  
}
```

Use `settings.defaultPage`, not `savedSettings.defaultPage`, because the user is previewing the selected option.

11. How your JSX changes

Replace things like this:

```
checked={largeText}
```

with this:

```
checked={settings.largeText}
onChange={() =>
  setSettings((current) => ({
    ...current,
    largeText: !current.largeText,
  })))
}
```

Your summary becomes:

```
<p><strong>Large Text:</strong> {settings.largeText ? "On" : "Off"}</p>
<p><strong>High Contrast:</strong> {settings.highContrast ? "On" : "Off"}</p>
<p><strong>Reduced Motion:</strong> {settings.reducedMotion ? "On" : "Off"}</p>
<p><strong>Event Reminders:</strong> {settings.eventReminders ? "On" : "Off"}</p>
<p><strong>Society Alerts:</strong> {settings.societyAlerts ? "On" : "Off"}</p>
<p><strong>Helpdesk Updates:</strong> {settings.helpdeskUpdates ? "On" : "Off"}</p>
<p><strong>Default Page:</strong> {settings.defaultPage}</p>
```

Your save button stays like this:

```
<button
  type="button"
  className={styles.saveButton}
  onClick={handleSaveSettings}
  disabled={savingSettings || !user}
>
  {savingSettings ? "Saving..." : "Save Settings"}
</button>
```

The logic in plain English

When the page loads, it asks the database:

What settings has this logged-in user saved?

Those saved settings are copied into the page state.

When the user clicks a checkbox, only the page state changes:

Preview changes now.
Do not touch the database.

When the user clicks **Save Settings**, the page sends the current preview settings to:

```
/api/settings
```

The API checks whether the user is logged in.

If they are logged in, it saves.

If they are not logged in, it rejects the request.

That gives you exactly this:

Previewable immediately ☒
Only saved after clicking Save ☒
Only saved after logging in ☒

